

SevaSetu — Frontend Specification Document

Product: SevaSetu

Tagline: Right Volunteer. Right Place. Right Impact.

Version: 1.0

Reference: SevaSetu PRD v2.0 + SevaSetu UI Reference

Frontend: React.js + Vite + Tailwind CSS

Backend: Node.js + Express.js REST API

Primary Database: Supabase

Agent State / Real-Time Agent Data: Firebase

Authentication: Clerk

Deployment: Vercel

1. Product UI Direction

SevaSetu is an intelligent volunteer operations platform connecting NGOs, volunteers, independent volunteers, donors/public users and communities.

The frontend must communicate three things immediately:

1. **Trust** — volunteer records and NGO information are transparent.
2. **Intelligence** — matching, scheduling and event guidance are assisted by AI.
3. **Community** — the product should feel human rather than like an enterprise management system.

The PRD establishes the product philosophy:

Agent recommends → Human decides.

The UI must therefore never make an AI recommendation look like an automatic irreversible decision.

2. Core User Flow

The primary visual/product flow is:

Need → Match → Schedule → Deploy → Track → Learn

Need

NGO creates an event and defines:

- Required skills
- Certifications
- Experience
- Volunteer count
- Location

- Date/time
- Shift requirements
- Event type

Match

The system recommends volunteers based on:

- Skills
- Certifications
- Availability
- Experience
- Distance
- Reliability

Schedule

The system generates:

- Shifts
- Volunteer rosters
- Primary assignments
- Backup assignments
- Conflict warnings

Deploy

The NGO can:

- Send emergency alerts
- Receive confirmations
- Review backup candidates
- Deploy confirmed volunteers

Track

The platform records:

- Attendance
- Task completion
- Service hours
- Assignment outcomes

Learn

The system produces:

- Karma
 - Reliability
 - Engagement analytics
 - Retention insights
 - Event intelligence
-

3. Design System

3.1 Brand Personality

The interface should feel:

- Trustworthy
- Human
- Clean
- Modern
- Helpful
- Community-focused
- AI-enabled without feeling overly technical

Avoid:

- Aggressive neon AI aesthetics
 - Excessive gradients
 - Dark futuristic dashboards
 - Excessive animations
 - Overly corporate enterprise styling
-

4. Color Palette

The reference design uses a predominantly white/light interface with deep teal-green branding and small colorful accents.

Primary Colors

Primary Green

#0B8F68

Use for:

- Primary buttons
- Active navigation
- Success states
- Brand elements
- Important actions

Dark Teal

#06343B

Use for:

- Main headings
- Hero sections
- Footer
- High-contrast panels
- Important text

Deep Green

#087A5A

Use for:

- Hover states
- Secondary emphasis
- Active indicators

Secondary Colors

Mint

#DDF5EA

Use for:

- Success backgrounds
- AI recommendation panels
- Volunteer status
- Positive information cards

Light Green

#EAF8F2

Use for:

- Section backgrounds
- Filters
- Information containers

Soft Blue

#EAF4FB

Use for:

- Information states
- Scheduling
- Analytics
- Location information

Soft Orange

#FFF1D8

Use for:

- Warnings
- Pending actions
- Emergency preparation notices

Soft Red

#FDECEC

Use for:

- Errors
- No-show warnings
- Failed operations
- Critical alerts

Neutral Palette

Background

#F8FAF9

Surface

#FFFFFF

Border

#DDE4E1

Primary Text

#122326

Secondary Text

#526467

Muted Text

#7C8B8D

5. Typography

Primary Font

Use:

Inter

Fallback:

system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif

Inter should be used for the entire application.

Typography Scale

H1

32–48px

Weight: 700

Used for:

- Landing page hero
- Major dashboard titles

H2

24–32px

Weight: 700

Used for:

- Section titles
- Dashboard sections

H3

18–22px

Weight: 600

Used for:

- Cards
- Event titles
- Feature headings

Body

14–16px

Weight: 400

Small

12–14px

Used for:

- Metadata
- Timestamps
- Labels
- Status indicators

6. Layout System

Use a 12-column responsive grid.

Desktop

Maximum content width:

1280px

Page padding:

32px

Tablet

Page padding:

24px

Mobile

Page padding:

16px

Spacing Scale

Use Tailwind spacing consistently:

- 4px

- 8px
- 12px
- 16px
- 24px
- 32px
- 48px
- 64px

Avoid arbitrary spacing unless necessary.

7. Border Radius

The reference design uses rounded UI components.

Small

8px

Inputs, badges and small elements.

Medium

12px

Cards and panels.

Large

16px

Major dashboard cards.

Extra Large

24px

Hero cards and major CTA containers.

Pills

9999px

Buttons and status badges where appropriate.

8. Shadows

Use subtle shadows only.

Card

0 2px 10px rgba(6,52,59,0.06)

Hover

0 6px 20px rgba(6,52,59,0.10)

Avoid heavy shadows.

9. Button System

Primary Button

Background:

#0B8F68

Text:

White

Example:

Join as a Volunteer

Register for Event

Create Event

Secondary Button

White background with:

#0B8F68 border

Used for:

- Learn More
 - View Details
 - Save
 - Cancel
-

Destructive Button

Use red only for destructive actions.

Examples:

- Delete Event
 - Remove Volunteer
 - Cancel Deployment
-

Emergency Button

Emergency actions should be visually distinct.

Example:

Send Emergency Alert

Use a warning/red treatment.

The UI must require confirmation before triggering an emergency broadcast.

10. Input Components

All inputs should have:

- 1px border
- 10–12px radius
- 12–14px internal padding
- Visible focus state
- Clear labels
- Helpful validation messages

Example:

Required Skills

[First Aid] [Medical] [+ Add Skill]

11. Cards

Cards are a core UI component.

Use cards for:

- Events
- Volunteers
- NGOs
- Analytics
- Agent recommendations
- Transparency information
- Karma
- Reliability

Standard structure:

Icon / Image

Title

Short description

Metadata

Primary action

Example:

Tree Plantation Drive

Green Earth NGO

May 3 • 7:30 AM

Within 10 km

[View Details](#)

12. Status Badges

Success

Active

Confirmed

Verified

Warning

Pending

Waitlisted

Needs Attention

Error

Cancelled

Failed

No Show

Information

Public

Emergency

Backup

13. Navigation

Desktop Navigation

Logo:

SevaSetu

Main links:

- Home
- For Volunteers
- For NGOs
- For Donors
- About

Right side:

- Login
 - Get Started
-

14. Landing Page

The reference image establishes the landing page as a strong hero-driven page.

Hero

Headline:

Right Volunteer.

Right Place.

Right Impact.

Supporting text should explain that SevaSetu connects NGOs, volunteers and donors using intelligent matching, scheduling and transparent impact tracking.

Primary CTA:

Join as a Volunteer

Secondary CTA:

Partner as an NGO

Impact Metrics

Show four compact statistics:

- Active Volunteers
- NGOs
- Events Completed

- Community Impact

These numbers should be dynamic when real production data exists.

15. How SevaSetu Works

Display six steps:

01 — Need

NGO creates an event and requirements.

02 — Match

System finds suitable volunteers.

03 — Schedule

Roster and shifts are generated.

04 — Deploy

Volunteers are confirmed and deployed.

05 — Track

Attendance and task completion are recorded.

06 — Learn

Karma, reliability and analytics are updated.

16. Volunteer Dashboard

The volunteer dashboard should prioritize opportunities and personal impact.

Top Section

Show:

- Service Hours
- Events Participated
- Karma Points
- Achievements

Recommended Events

Event cards should be ranked based on:

- Skills
- Interests
- Availability

- Distance
- Reliability

Each recommendation should explain why it was recommended.

Example:

92% Match

Skills match

Within preferred distance

Available during event

17. NGO Dashboard

The NGO dashboard should prioritize operational control.

KPI Cards

- Active Events
- Total Volunteers
- Roster Completion
- NGO Transparency Indicators

Upcoming Events

Show:

- Event name
- Date
- Volunteer requirement
- Filled positions
- Roster status

Quick Actions

- Create Event
 - Manage Volunteers
 - View Rosters
 - Mark Attendance
 - View Analytics
 - Open Agent
-

18. Event Details Page

The Event page is one of the most important screens because the PRD defines the Event as the central entity.

Display:

Event Header

- Event image
- Event name
- NGO
- Category
- Date
- Time
- Location
- Distance

Requirements

- Skills
- Certifications
- Experience
- Age requirements if applicable
- Availability requirements

Capacity

Example:

12 / 20 volunteers confirmed

Event Actions

Volunteer:

Register for Event

NGO:

Manage Event

19. Volunteer Registration Flow

Flow:

Event → View Requirements → Register → Confirmation

Registration should validate:

- Availability
- Event capacity
- Existing assignment conflicts
- Required skills/certifications where applicable

Possible states:

- Registered
- Accepted
- Waitlisted
- Rejected
- Cancelled

20. Matching UI

The matching interface should show ranked candidates rather than simply saying "AI selected this person."

Example:

Volunteer Match

Kundan — 94% Match

- Skill Match: 40/40
- Availability: 20/20
- Distance: 14/15
- Experience: 12/15
- Certification: 8/10

Reliability: 91/100

Action:

Assign

The UI must make clear that the NGO/coordinator makes the final decision.

21. Scheduling UI

Display a timeline or shift table.

Shift	Required	Assigned	Status
-------	----------	----------	--------

08:00–12:00	10	10	Filled
-------------	----	----	--------

Shift	Required	Assigned	Status
12:00–16:00	10	8	Needs 2
16:00–20:00	8	8	Filled

Highlight:

- Conflicts
- Unfilled positions
- Backup volunteers
- Over-capacity assignments

22. Emergency UI

Emergency functionality must be treated differently from normal events.

Display:

Emergency Response

Location

Required Volunteers

Required Skills

Current Confirmations

Backup Candidates

Deployment Status

Example:

8 / 12 Confirmed

4 Backup Candidates

State Flow

Alerted → Confirmed → Deployed

The frontend must never treat "confirmed" as "deployed."

Only an authorized NGO coordinator/admin can deploy confirmed volunteers.

23. Event Intelligence Agent

The Agent is a recommendation and guidance system.

The reference UI shows the Agent as a dedicated assistant panel.

Agent Navigation

Suggested modes:

- Event Guidance
 - Historical Analysis
 - Preparation
 - Risk & Emergency
 - Volunteer Assistant
 - NGO Assistant
-

Agent Response Structure

Every important response should have:

Recommendation

What the Agent recommends.

Basis

Why it recommends it.

Historical Context

If historical event data was used.

Uncertainty

If the recommendation is uncertain.

Example:

Preparation Recommendation

For this event, consider having:

- 4 first-aid trained volunteers
- Drinking water
- Basic medical supplies
- Weather protection

Basis: Similar events had increased demand for first-aid support.

The Agent must not:

- Create emergencies
- Broadcast emergencies
- Finalize rosters

- Change attendance
 - Award Karma directly
 - Override NGO decisions
 - Make irreversible operational decisions
-

24. NGO Transparency Profile

The transparency profile should be publicly accessible.

Display:

- NGO name
- Description
- Causes
- Location
- Established year
- Events conducted
- Volunteer participation
- Activity indicators
- Transparency information
- Public reports where available

Important UI rule:

Do not label an NGO as:

"Good NGO"

"Bad NGO"

"Fraudulent NGO"

Instead use neutral platform-derived indicators.

Example:

Activity Indicators

Events Conducted: 120+

Volunteers Participated: 15K+

Recent Activity: Active

25. Independent Volunteer Discovery

Independent volunteers can discover public NGO events without being permanently affiliated with an NGO.

Discovery UI:

Search

Search by:

- Event
- NGO
- Skill
- Cause
- Location

Filters

- Environment
- Health
- Education
- Community
- Emergency
- Distance
- Date

Event cards should show:

- NGO
- Event
- Location
- Date
- Cause
- Distance
- Available positions

26. Karma / Impact Profile

Karma represents verified community contribution.

Display:

Karma Points

1,250

Service Hours

45

Events Participated

12

Achievements

- Community Helper
- Emergency Responder
- 10 Events Completed

Karma must not be presented as:

- Government certification
- Guaranteed employment credential
- Guaranteed NGO position

Karma and Reliability must remain separate concepts.

27. Reliability UI

Reliability should be transparent.

Display factors such as:

- Completed assignments
- Cancellations with notice
- No-shows
- Recent participation
- Task completion
- Punctuality where tracked

Example:

Reliability: 91/100

Excellent consistency

The UI should explain that reliability is an operational matching signal rather than an absolute judgment of the individual.

28. Analytics Dashboard

NGO analytics should include:

Volunteer Growth

Line chart

Service Hours

Total and monthly comparison

Event Completion

Completed vs planned

Participation

Attendance trends

Skill Gaps

Required vs available skills

Engagement

Returning vs new volunteers

Retention

Volunteer participation over time

29. Responsive Design

Desktop

Use:

- Sidebar dashboards
- Multi-column cards
- Large analytics panels
- Wide event pages

Tablet

Use:

- Collapsible sidebar
- Two-column cards

Mobile

Use:

- Bottom navigation
- Single-column cards

- Sticky primary CTA
- Compact analytics
- Full-screen Agent interface

The reference image already establishes a mobile-oriented SevaSetu view, so mobile should not simply be treated as a shrunk desktop layout.

30. Frontend Component Architecture

Recommended structure:

src/

```

├── app/
│   ├── routes/
│   ├── providers/
│   └── layouts/
│
├── components/
│   ├── ui/
│   │   ├── Button
│   │   ├── Input
│   │   ├── Modal
│   │   ├── Card
│   │   ├── Badge
│   │   ├── Avatar
│   │   ├── Tabs
│   │   ├── Table
│   │   └── Toast
│   │
│   ├── event/
│   ├── volunteer/
│   ├── ngo/
│   ├── emergency/
│   └── agent/

```

- | |— analytics/
- | |— transparency/
- |
- |— pages/
- | |— Landing
- | |— Login
- | |— VolunteerDashboard
- | |— NGODashboard
- | |— EventDetails
- | |— EventCreate
- | |— VolunteerDiscovery
- | |— TransparencyProfile
- | |— Analytics
- | |— Agent
- |
- |— services/
- | |— api/
- | |— auth/
- | |— maps/
- | |— notifications/
- | |— agent/
- |
- |— hooks/
- |— store/
- |— utils/
- |— types/
- |— assets/
- |— styles/

31. API Architecture

The frontend communicates with the Express REST API.

Base URL:

/api/v1

Authentication:

Authorization: Bearer <Clerk Session Token>

The backend validates the Clerk session before processing protected requests.

32. Core API Endpoints

Events

GET /events

GET /events/:id

POST /events

PUT /events/:id

POST /events/:id/publish

POST /events/:id/close

Create Event

Request:

```
{
  "title": "Medical Relief Camp",
  "description": "Community medical support camp",
  "cause": "Health",
  "location": {},
  "start": "2026-10-10T09:00:00",
  "end": "2026-10-10T14:00:00",
  "capacity": 30,
  "visibility": "public"
}
```

Response:

```
{
  "id": "event_123",
  "status": "draft",
```

```
"created": true
}
```

33. Event Requirements API

GET /events/:id/requirements

POST /events/:id/requirements

PUT /events/:id/requirements/:requirementId

Example:

```
{
  "role": "Medical Volunteer",
  "headcount": 5,
  "requiredSkills": ["First Aid"],
  "requiredCertifications": ["First Aid Certification"],
  "minExperience": 1
}
```

34. Volunteer APIs

GET /volunteers/me

PUT /volunteers/me

GET /volunteers/:id

GET /volunteers/discover

Volunteer profile data includes:

- Skills
- Certifications
- Experience
- Location
- Availability
- Causes
- Preferences

Sensitive personal information must not be exposed through public discovery.

35. Registration APIs

POST /events/:id/apply

GET /events/:id/applications

PATCH /applications/:id

Possible status:

interested

accepted

waitlisted

rejected

cancelled

36. Matching APIs

POST /events/:id/match

GET /events/:id/matches

Response:

```
{
  "eventId": "event_123",
  "matches": [
    {
      "volunteerId": "vol_123",
      "matchScore": 94,
      "reasons": [
        "Required skill matched",
        "Available during shift",
        "Within preferred distance",
        "Relevant certification"
      ],
      "reliabilityScore": 91
    }
  ]
}
```

The frontend must display the reasons alongside the score.

37. Roster APIs

POST /events/:id/roster/generate

GET /events/:id/roster

POST /events/:id/roster/finalize

The frontend must display:

- Primary assignments
- Backup assignments
- Conflicts
- Unfilled positions

Finalization requires authorized NGO staff.

38. Attendance APIs

POST /events/:id/attendance/checkin

POST /events/:id/tasks/:taskId/complete

POST /events/:id/close

Attendance updates can subsequently contribute to:

- Reliability
 - Karma
 - Analytics
-

39. Karma API

GET /karma/:volunteerId

Response:

```
{  
  "volunteerId": "vol_123",  
  "karma": 1250,  
  "serviceHours": 45,  
  "eventsParticipated": 12,  
  "achievements": []  
}
```

}

The frontend should treat Karma as a verified contribution history rather than a generic reputation score.

40. NGO Transparency API

GET /ngos/:id/transparency

PUT /ngos/:id/transparency

Public GET:

Returns public organization and activity indicators.

PUT:

Restricted to authorized NGO users.

41. Analytics APIs

GET /ngos/:id/analytics

GET /admin/analytics

Dashboard metrics should be retrieved through aggregated API responses rather than exposing raw volunteer records.

42. Emergency APIs

POST /emergencies

POST /emergencies/:id/alert

POST /emergencies/:id/respond

GET /emergencies/:id/status

POST /emergencies/:id/backups

POST /emergencies/:id/deploy

POST /emergencies/:id/checkin

Important State Model

Created

↓

Alerted

↓

Confirmed

↓

Deployed

↓

Checked-in

↓

Completed

A volunteer confirming availability does not mean deployment.

43. Emergency Real-Time Integration

Supabase is the source of truth for emergency records.

The real-time layer mirrors:

- Alert state
- Confirmation count
- Response changes

If real-time delivery fails, the frontend must fall back to API polling.

Frontend behavior:

Subscribe to emergency updates

↓

Receive confirmation

↓

Update live count

↓

Refresh backup recommendations

Never rely exclusively on the client-side real-time state.

44. Agent APIs

POST /agent/sessions

POST /agent/sessions/:id/messages

GET /agent/sessions/:id

Message request:

{

```
"question": "What should I prepare for this event?",
"mode": "event_guidance"
}
```

Backend constructs a controlled context from:

- Event
- Requirements
- Aggregated historical event information
- NGO profile summary

The frontend receives a structured Agent response.

Example:

```
{
  "answer": "Consider additional first-aid coverage...",
  "basis": [
    "similar_event_123",
    "event_requirement_456"
  ],
  "uncertainty": "Moderate"
}
```

The frontend should display the basis and uncertainty when supplied.

45. Clerk Integration

Purpose

Clerk handles:

- Authentication
- Login
- Signup
- Session management
- Identity

Frontend responsibilities:

User Login

↓

Clerk Session

↓

Session Token

↓

Express API

↓

Authorization

Protected frontend routes should check authentication before rendering private dashboards.

The PRD specifically uses Clerk authentication with authorization enforced through Express, with database RLS as an additional defense.

46. Supabase Integration

Supabase is the **primary source of truth**.

Store:

- Users
- Volunteer profiles
- Skills
- Certifications
- Availability
- NGOs
- NGO memberships
- Events
- Requirements
- Shifts
- Applications
- Match runs
- Assignments
- Attendance
- Tasks
- Reliability
- Karma

- Transparency
- Emergency records
- Notifications

Frontend should generally access these through the Express API rather than bypassing backend authorization for business operations.

47. Firebase Integration

Firebase is used for the Agent's context/state and real-time agent-related data.

Use it for:

- Agent session state
- Agent conversation state where required
- Temporary agent context
- Real-time Agent UI state

Do not duplicate core business records into Firebase.

Rule:

Supabase = authoritative business data

Firebase = Agent state / real-time Agent context

48. Maps API Integration

Purpose:

- Event location
- Distance calculation
- Volunteer proximity
- Location-based discovery
- Emergency volunteer targeting

The backend should perform sensitive distance calculations.

The client should receive:

```
{  
  "distanceKm": 7.4  
}
```

rather than exposing another volunteer's precise coordinates.

The exact Maps provider and provider-specific endpoint should remain configurable because the PRD specifies a Maps API but does not lock the implementation to a particular vendor.

49. Email API

Email is a secondary/fallback notification channel.

Use for:

- Event invitations
- Assignment notifications
- Shift reminders
- Emergency fallback notifications
- Schedule changes
- Participation confirmations

Emergency principle:

In-app real-time alert first → Email fallback second

The exact email provider and endpoint should be configured through environment variables rather than hard-coded into frontend code.

Frontend should call the SevaSetu backend notification endpoint rather than directly exposing email credentials.

50. n8n Integration

n8n is the workflow automation layer.

Potential workflows:

Event Created

↓

Notification Workflow

↓

Email / Internal Notification

Shift Reminder

↓

n8n

↓

Notification

Emergency fallback condition

↓

n8n

↓

Email notification

n8n should not become the source of truth for business data.

51. AI / LLM Integration

The Agent service uses Python and a selected LLM provider.

Potential models:

- ChatGPT
- Gemini
- Hugging Face models

The frontend should never directly expose LLM API keys.

Architecture:

React

↓

Express

↓

Python Agent Service

↓

LLM Provider

↓

Structured Agent Response

↓

Express

↓

React

The backend should validate Agent output before returning it to the user.

52. AI Safety UI

Every Agent recommendation should visually communicate that it is a recommendation.

Recommended label:

AI Recommendation

Optional:

Human approval required

For emergency operations:

The Agent cannot create, broadcast or deploy an emergency.

This reinforces the PRD principle:

Agent recommends, human decides.

53. Error Handling

Every API request must support:

Loading

Show skeleton or spinner.

Success

Show confirmation.

Validation Error

Show field-specific explanation.

Unauthorized

Redirect to login or show permission error.

Forbidden

Show:

"You don't have permission to perform this action."

Not Found

Show:

"This event could not be found."

Server Error

Show:

"Something went wrong. Please try again."

Network Error

Show:

"Unable to connect. Check your internet connection."

54. Emergency Error Handling

Emergency operations require stricter handling.

If alert fails:

Emergency alert was not sent. Please retry.

If confirmation endpoint times out:

Do not automatically submit again unless the request is idempotent.

Display:

Checking emergency status...

Then retrieve current state.

If real-time connection fails:

Live updates temporarily unavailable. Refreshing status automatically.

55. Accessibility

The frontend should support:

- Keyboard navigation
- Visible focus states
- Semantic HTML
- Proper button labels
- Accessible form labels
- Sufficient contrast
- Screen-reader-friendly status messages
- Non-color-only status indicators

Do not communicate:

"Green = confirmed"

without also showing:

"Confirmed"

56. Animation

Animation should be subtle.

Use animation for:

- Page transitions
- Card hover
- Modal entrance
- Agent message appearance
- Progress indicators
- Dashboard chart loading

Avoid animation for:

- Critical emergency actions
- Important data
- Large background elements

Recommended animation duration:

150–250ms

57. Frontend State Management

Separate:

Server State

- Events
- Volunteers
- NGO data
- Applications
- Analytics
- Emergency status

UI State

- Modal open/closed
- Filters
- Sidebar
- Tabs

Agent State

- Session
- Current conversation

- Streaming response
- Agent mode

Avoid duplicating server state unnecessarily in global UI state.

58. Security Rules for Frontend

Never store in frontend code:

- API secrets
- Clerk secret key
- LLM API keys
- Supabase service-role key
- Firebase admin credentials
- Maps server credentials
- Email API secret

Use environment variables only for public/client-safe values.

All sensitive operations must go through the backend.

59. Recommended Environment Variables

VITE_API_BASE_URL=

VITE_CLERK_PUBLISHABLE_KEY=

SUPABASE_URL=

SUPABASE_ANON_KEY=

SUPABASE_SERVICE_ROLE_KEY=

FIREBASE_PROJECT_ID=

FIREBASE_API_KEY=

MAPS_API_KEY=

EMAIL_API_KEY=

N8N_WEBHOOK_URL=

OPENAI_API_KEY=

GEMINI_API_KEY=

HUGGINGFACE_API_KEY=

AGENT_SERVICE_URL=

AGENT_SERVICE_SECRET=

Important:

Any variable beginning with VITE_ may be exposed to the browser. Never place a server secret in a VITE_ variable.

60. API Integration Rule

The frontend should follow:

UI

↓

React Service Layer

↓

Express REST API

↓

Authorization

↓

Business Logic

↓

Supabase / Firebase / External Service

Do not create this architecture:

UI

↓

External APIs directly

for protected or secret operations.

61. Frontend Folder Responsibilities

/components

Reusable visual components.

/pages

Complete screens.

/services

API communication.

/hooks

Reusable frontend behavior.

/store

Global client state where necessary.

/types

Shared TypeScript interfaces.

/utils

Pure helper functions.

/layouts

Dashboard and public layouts.

62. Important Frontend Data Types

```
type UserRole =
```

```
  | "ngo_admin"
```

```
  | "ngo_coordinator"
```

```
  | "independent_volunteer"
```

```
  | "affiliated_volunteer"
```

```
  | "donor"
```

```
  | "platform_admin";
```

```
type EventStatus =
```

```
  | "draft"
```

```
  | "published"
```

```
  | "active"
```

```
| "completed"
| "cancelled";
type ApplicationStatus =
| "interested"
| "accepted"
| "waitlisted"
| "rejected"
| "cancelled";
type EmergencyVolunteerState =
| "alerted"
| "confirmed"
| "deployed";
```

63. Golden UI Principle

Every major screen should answer one question immediately.

Volunteer Dashboard

"What should I do next?"

NGO Dashboard

"What needs my attention?"

Event Page

"Can I participate?"

Matching Page

"Who should we assign and why?"

Emergency Page

"Who is available right now?"

Agent

"What should I know before acting?"

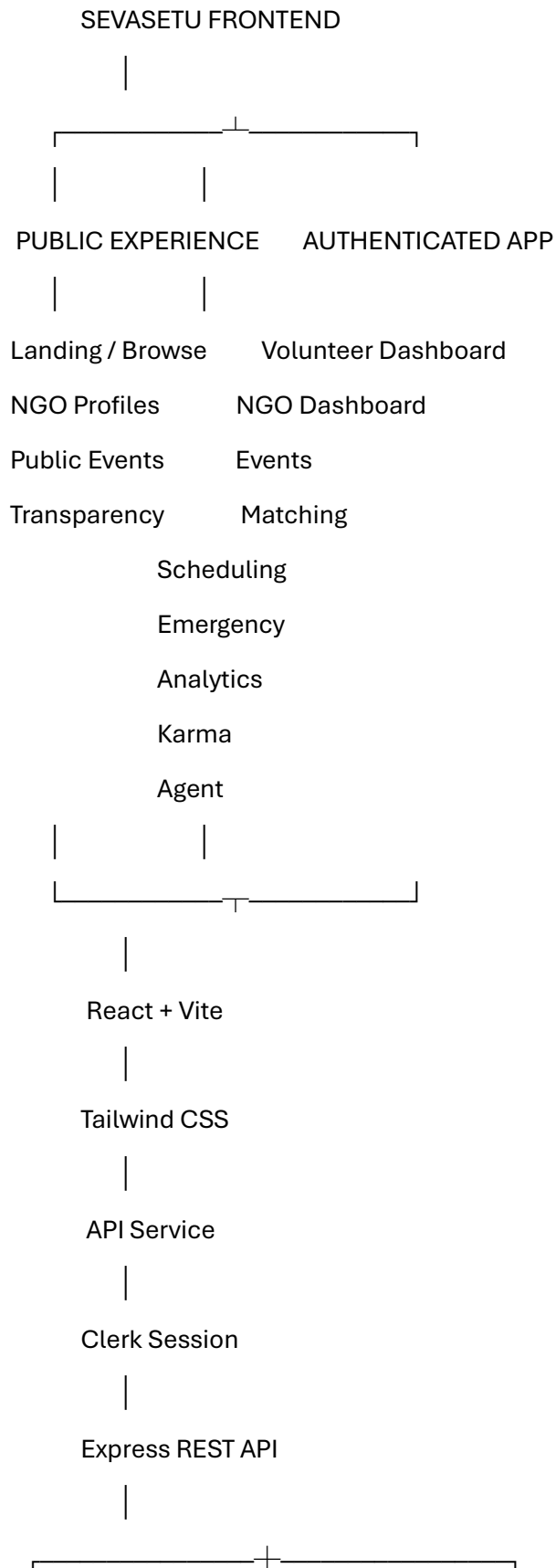
Transparency Profile

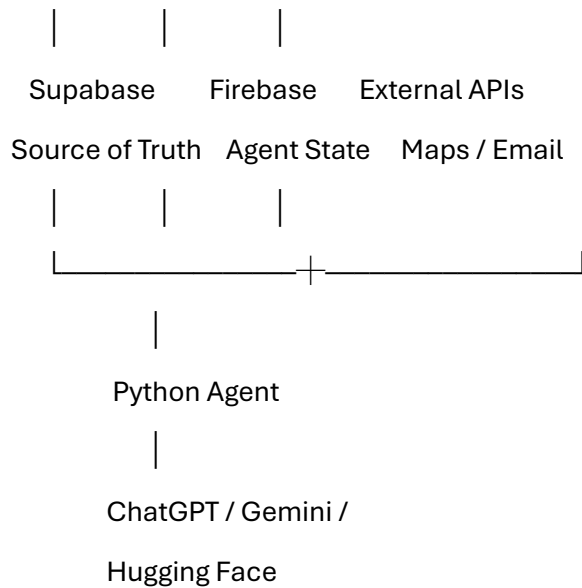
"What has this NGO actually been doing?"

Analytics

"What is improving or getting worse?"

64. Final Frontend Architecture





65. Core Implementation Rule

The frontend must always preserve the distinction between:

Recommendation vs Decision

Confirmation vs Deployment

Karma vs Reliability

Transparency Indicator vs NGO Certification

Supabase Source of Truth vs Firebase Agent State

These distinctions are fundamental to the SevaSetu product architecture and should be reflected directly in the UI, API contracts and state management.

66. MVP Priority

For the hackathon implementation, prioritize:

Must Have

- Authentication
- Volunteer profile
- NGO profile
- Event creation
- Public event discovery
- Event registration
- Matching

- Roster generation
- Attendance
- Karma
- Reliability
- NGO transparency
- Emergency alert flow
- Event Intelligence Agent
- Basic analytics

Secondary

- Advanced analytics
- Advanced Agent capabilities
- Advanced notification automation
- Advanced matching controls

Future

- Persistent specialized emergency groups
- QR/geo attendance verification
- SMS/WhatsApp
- Advanced automation
- Additional ecosystem integrations

The PRD explicitly positions the MVP as a focused subset of the broader scalable vision.

67. Design North Star

Every SevaSetu interface should reinforce:

Right Volunteer.

Right Place.

Right Impact.

The interface should make complex volunteer operations feel simple while keeping humans in control of important decisions.